

[13] pgvector: Open-Source Vector Similarity Search for PostgreSQL

개발자: Andrew Kane

유형: PostgreSQL 확장(extension), GitHub 오픈소스

활동 기간: 2021년~현재

GitHub 스타: 12,000+ (2025년 기준)

역할군: (B) 대상 시스템

요약

pgvector는 **가장 널리 사용되는 범용 벡터 DB**로, PostgreSQL에 벡터 데이터 타입과 유사도 검색 기능을 추가한다. Exqutor의 주요 실험 플랫폼이며, PostgreSQL의 완전한 SQL 기능을 활용하면서도 벡터 검색을 지원한다.

pgvector의 장점:

1. **전체 SQL 기능 지원:** 조인, 집계, 서브쿼리, 윈도우 함수, CTE 등 모든 SQL 기능 사용 가능
2. **기존 PostgreSQL 생태계와 호환:** pg_dump, 복제, 백업 등 기존 도구 그대로 사용
3. **MVCC 기반 트랜잭션:** 벡터 데이터도 일관된 트랜잭션 처리 가능
4. **다양한 거리 함수 지원:** 유클리드 거리, 코사인 거리, 내적 등
5. **오픈소스:** GitHub에서 자유롭게 수정·배포 가능

하지만 pgvector는 **벡터 검색의 선택도를 항상 33.3%로 고정 추정**한다는 치명적 한계가 있으며, 이것이 최적화 기회를 상당히 제한한다. Exqutor의 ECQO가 pgvector에서 최대 1,000배 속도 향상을 달성한 주요 이유이다.

상세분석

13.1 핵심 특징: 벡터 데이터 타입과 연산자

pgvector는 PostgreSQL의 **확장(extension)**으로 구현되며, 기본적으로 다음을 추가한다:

벡터 데이터 타입:

```
CREATE TABLE items (
  id SERIAL PRIMARY KEY,
  name TEXT,
  embedding vector(768) -- 768차원 벡터
);

INSERT INTO items VALUES (1, 'product_a', '[0.1, 0.2, 0.3, ...] '::vector);
```

이렇게 벡터를 일반 컬럼처럼 취급할 수 있으므로, 벡터를 포함한 복잡한 쿼리를 작성할 수 있다.

유사도 검색 연산자:

pgvector는 3가지 거리 함수를 지원한다:

연산자	거리 함수	수식	용도
<code><-></code>	유클리드 거리 (L2)	$\sqrt{(\sum (a_i - b_i)^2)}$	일반적인 거리 측정
<code><=></code>	코사인 거리	$1 - (A \cdot B) / (\ A\ \ B\)$	방향(의미) 유사도
<code><#></code>	내적 (inner product)	$A \cdot B$	추천 시스템 등

활용 예시:

```
-- 유클리드 거리로 상위 10개 찾기
SELECT id, name, embedding <-> query_vector AS distance
FROM items
ORDER BY distance
LIMIT 10;

-- 코사인 거리가 0.2 이하인 것만
SELECT * FROM items
WHERE (embedding <=> query_vector) < 0.2
LIMIT 100;
```

13.2 인덱스 지원: HNSW와 IVFFlat

pgvector는 두 가지 벡터 인덱스를 지원한다.

HNSW 인덱스 (Hierarchical Navigable Small World):

```
CREATE INDEX ON items USING hnsw (embedding vector_l2_ops)
WITH (m=16, ef_construction=64);
```

파라미터 의미:

- `m=16` : 각 노드가 최대 16개 이웃을 가짐. 커질수록 정확도 높지만 메모리 증가.
- `ef_construction=64` : 인덱스 구축 시 탐색 깊이. 커질수록 구축 시간 증가.

HNSW는 **동적 인덱스**로, 데이터 삽입/삭제 시에도 인덱스가 자동으로 업데이트된다.

IVFFlat 인덱스 (Inverted File with Flat Quantization):

```
CREATE INDEX ON items USING ivfflat (embedding vector_l2_ops)
WITH (lists=100);
```

파라미터 의미:

- `lists=100` : 벡터 공간을 100개 클러스터로 분할

IVFFlat는 **정적 인덱스**로, 데이터 변경 후 인덱스를 재구축해야 한다. 대신 메모리 효율이 좋고 구축이 빠르다.

13.3 구현 세부사항: PostgreSQL 통합의 장단점

pgvector의 가장 큰 장점:

pgvector는 PostgreSQL의 모든 인프라를 그대로 사용하므로:

1. **전체 SQL 기능**: 조인, 집계, 서브쿼리, 윈도우 함수, CTE, 트리거, 저장프로시저 등 모든 SQL 기능이 벡터 컬럼과 함께 작동한다.

```
sql -- 벡터 검색 + 집계 + 조인 SELECT p.category, COUNT(*) AS count, AVG(p.price) AS avg_price
FROM items p JOIN similar_items s ON p.id = s.item_id WHERE (p.embedding <=> query_vector) <
0.1 GROUP BY p.category ORDER BY count DESC;
```

1. **EXPLAIN ANALYZE**: 실행 계획을 확인할 수 있어 성능 최적화가 용이하다.
2. **생태계 호환성**: pg_dump로 벡터 데이터까지 백업 가능, 복제(replication)도 그대로 작동한다.
3. **MVCC 기반 트랜잭션**: 벡터 데이터도 일관된 트랜잭션 처리로 데이터 무결성 보장.

pgvector의 근본적 한계:

벡터는 PostgreSQL의 **사용자 정의 타입(custom type)**으로 구현되므로:

1. **버퍼 풀 오버헤드**: 모든 벡터 접근이 공유 버퍼 풀을 경유한다. HNSW 그래프 탐색 시 매번 래치(latch)를 획득해야 하므로, 이것이 성능의 주요 병목이 된다 (논문 [20]에서 지적).
2. **튜플 헤더 파싱**: 각 벡터를 읽을 때마다 PostgreSQL의 튜플 헤더(트랜잭션 정보, 가시성 정보)를 파싱해야 하는데, 벡터 검색에는 이 정보가 불필요한 오버헤드다.
3. **공간 증폭(Space Amplification)**: HNSW 인덱스가 PostgreSQL의 8KB 페이지 단위로 저장되므로, 기본 테이블보다 훨씬 큰 공간을 차지한다 (2~3배).

13.4 선택도 추정의 치명적 한계: 고정 33.3%

pgvector의 가장 심각한 한계:

pgvector의 핵심 한계는 벡터 검색의 선택도를 **항상 33.3%(= 1/3)로 고정 추정**하는 것이다:

```
// pgvector 소스 코드 (pg_vector.c)
#define DEFAULT_SEL 0.333333
```

이 숫자는 아무런 근거 없는 기본값일 뿐, 실제 데이터 분포를 반영하지 않는다.

왜 이것이 문제인가?

PostgreSQL의 옵티마이저는 각 조건의 선택도(결과의 몇 %가 그 조건을 통과하는가)를 기반으로 **조인 순서, 조인 방식, 인덱스 사용 여부**를 결정한다.

예시 쿼리:

```
SELECT * FROM items
WHERE (embedding <=> query_vector) < 0.1  -- 벡터 조건
      AND category = 'electronics'         -- 스칼라 조건
ORDER BY price;
```

시나리오 1: 벡터 조건의 실제 선택도 = 0.1% (1000개 중 1개만 유사)

- 옵티마이저의 추정: 33.3%
- 실제 결과: 0.1%

이 경우 옵티마이저는 **HNSW 인덱스를 사용하지 않고 Sequential Scan**을 선택할 수 있다 (예상 결과가 많으니 풀 스캔이 빠르다고 판단). 하지만 실제로는 인덱스를 사용하는 것이 훨씬 빠르다.

시나리오 2: 벡터 조건의 실제 선택도 = 90% (거의 모든 데이터가 유사)

- 옵티마이저의 추정: 33.3%
- 실제 결과: 90%

이 경우 옵티마이저는 인덱스를 사용할 것으로 예상하지만, 실제로는 결과가 매우 많아서 Sequential Scan이 더 빠를 수 있다. 그럼에도 옵티마이저는 인덱스를 선택하여 불필요한 인덱스 탐색을 수행한다.

실제 성능 영향:

최적의 실행 계획과 실제 선택된 계획 사이에 **수십~수천 배의 성능 차이**가 발생할 수 있다. Exqutor는 이 문제를 직접 해결함으로써 pgvector에서 최대 **1,000배** 속도 향상을 달성했다.

13.5 본 논문과의 관계: Exqutor가 pgvector를 선택한 이유

Exqutor의 ECQO(Effective Cardinality-aware Query Optimization)는 pgvector에서 **PostgreSQL의 플래너 훅(planner hook)**을 확장하여 구현된다.

Exqutor의 해결책:

1. **플래너 훅 확장**: PostgreSQL이 쿼리 계획을 수립할 때, Exqutor가 벡터 조건을 가로챈다.
2. **실제 인덱스 탐색**: 벡터 조건의 정확한 카디널리티를 얻기 위해, HNSW 인덱스에 **실제 범위 검색을 실행**한다 (1~2ms 소요).
3. **정보 전달**: 얻은 카디널리티 정보를 PostgreSQL 옵티마이저에 전달한다.
4. **최적 계획 수립**: 옵티마이저가 정확한 카디널리티 정보를 바탕으로 최적 실행 계획을 수립한다.

이 접근법의 장점:

- pgvector에 최소한의 변경 (planner hook만 활용)
- 기존 PostgreSQL 인프라와 완전 호환
- 모든 유형의 벡터 조건(거리 임계값, top-k 등)을 지원

13.6 pgvector의 진화와 미래

pgvector는 지속적으로 개선되고 있다:

pgvector 0.7.0+ 개선사항:

- HNSW 인덱스 성능 크게 향상
- 동적 인덱스 구축 최적화
- 메모리 사용량 감소

하지만 **카디널리티 추정 문제는 여전히 미해결**이다. pgvector가 이를 자체적으로 해결하려면:

- 벡터 통계 수집 기능 추가 (ANALYZE 명령에 통합)
- 고차원 벡터의 거리 분포 모델링
- 옵티마이저에 벡터 전용 카디널리티 추정 로직 구현

이 모든 것이 복잡한 작업이므로, Exqutor 같은 **외부 확장 기반의 접근**이 더 현실적일 수 있다.

추가 제기 문제

pgvector가 기술적으로는 훌륭하지만, 산업계에서는 여전히 Milvus, Qdrant 같은 전문 벡터 DB를 선호하는 경향이 있다. 이유는:

1. 벡터 검색에 특화된 성능
2. 전문 벡터 DB의 더 나은 필터링 지원
3. PostgreSQL의 "기본 오버헤드"에 대한 우려

하지만 Exqutor의 연구는 pgvector의 가능성을 보여주며, 올바른 최적화 기법으로 이런 격차를 상당히 줄일 수 있음을 입증했다. 향후 pgvector가 벡터 통계를 추가로 지원하고 옵티마이저를 개선하면, 전문 벡터 DB와의 격차는 더욱 좁혀질 것으로 예상된다.